AISC PRACTICAL

Part B Code Handbook | Experiments 1–10

Practical-ready Python programs, explanations and visual path diagrams

New in this version: Search-tree diagrams for BFS, DFS and A* show the order in which nodes are explored and the final path. These make the search algorithms easier to visualize and remember.

Note: *Experiment 1 contains a PEAS write-up rather than executable code in the supplied Part B. Experiments 2–10 contain executable programs.*

Quick Index

Experiment 01: PEAS DESCRIPTORS FOR AN INTELLIGENT AGENT

Experiment 02: UNINFORMED SEARCH: BFS AND DFS

Experiment 03: INFORMED SEARCH: A* ALGORITHM

Experiment 04: SINGLE LAYER PERCEPTRON: AND AND OR GATES

Experiment 05: MULTI-LAYER PERCEPTRON: SUPERVISED LEARNING

Experiment 06: KOHONEN SELF-ORGANIZING MAP (KSOM)

Experiment 07: ASSOCIATIVE MEMORY: PART B CODE IMPLEMENTS BAM

Experiment 08: FUZZY MEMBERSHIP FUNCTIONS

Experiment 09: FUZZY CONTROL SYSTEM SIMULATION

Experiment 10: GENETIC ALGORITHM OPTIMIZATION

EXPERIMENT 01

PEAS DESCRIPTORS FOR AN INTELLIGENT AGENT

Aim: Identify an AI problem statement and describe its PEAS descriptors and task-environment properties.

Source note: Part B of the supplied Experiment 1 PDF contains a written PEAS answer for an Explainable Movie Recommender System; it does not contain an executable program.

Core concept

PEAS means Performance measure, Environment, Actuators and Sensors. It is used to specify an intelligent agent's task environment before designing the agent.

PEAS descriptors

PEAS	Movie Recommender Agent
Performance measure	Recommendation accuracy, relevance, explanation quality and user satisfaction.
Environment	MovieLens users, ratings, movie details and unrated movies.
Actuators	Display ranked recommendations and explanations; update recommendations when ratings arrive.
Sensors	Past ratings, browsing history and movie metadata.

Environment properties: Partially observable, stochastic, sequential, dynamic, discrete and multi-agent.

Memory line: *P = How well? E = Where? A = What can it do? S = What can it sense?*

EXPERIMENT 02

UNINFORMED SEARCH: BFS AND DFS

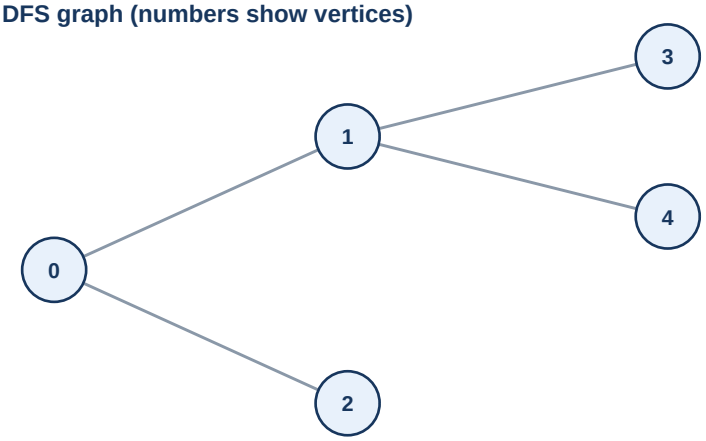
Aim: Implement uninformed search methods using graph traversal.

Source note: The supplied Part B code is Java and implements BFS and DFS using an adjacency list. The version below is a shorter Python equivalent with the same graph and traversal functionality.

Core concept

BFS explores level by level using a FIFO queue. DFS follows one branch deeply before backtracking, using recursion (call stack). Both use a visited set so a vertex is not processed repeatedly.

BFS / DFS graph (numbers show vertices)



BFS order: 0 → 1 → 2 → 3 → 4. It visits all nodes at one level before moving deeper. **DFS order:** 0 → 1 → 3 → 4 → 2. It follows one branch deeply, then backtracks. The highlighted idea is the traversal order; the graph itself is the same for both.

Practical code

```
from collections import deque

# Graph using an adjacency list
graph = {
    0: [1, 2],
    1: [3, 4],
    2: [],
    3: [],
    4: []
}

def bfs(start):
    visited = set([start])
    queue = deque([start])
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

    return order

def dfs(start):
    visited = set()
    order = []

    def visit(node):
        visited.add(node)
        order.append(node)

        for neighbour in graph[node]:
            if neighbour not in visited:
                visit(neighbour)
```

```
    visit(start)
    return order

print("BREADTH FIRST SEARCH:", bfs(0))
print("DEPTH FIRST SEARCH:  ", dfs(0))
```

Code explanation

• **Graph:** the dictionary stores each vertex and its neighbouring vertices. For example, 0: [1, 2] means 0 is connected to 1 and 2.

• **BFS:** add the start vertex to a queue; remove from the front; visit its unvisited neighbours and add them to the back.

• **DFS:** mark the current vertex visited, print/store it, then recursively visit each unvisited neighbour.

• **Visited set:** prevents revisiting a node and avoids infinite loops if the graph contains a cycle.

Easy memory line

BFS = **Queue** = FIFO = level by level. DFS = **Recursion/Stack** = go deep, then backtrack.

Input and expected output

Input: The graph and starting vertex are already set in the program. Start vertex = 0.

Expected output: BFS: [0, 1, 2, 3, 4]. DFS: [0, 1, 3, 4, 2].

How to run: *Save as exp2.py and run:* ***python exp2.py****. To change the graph, edit the graph dictionary.*

EXPERIMENT 03

INFORMED SEARCH: A* ALGORITHM

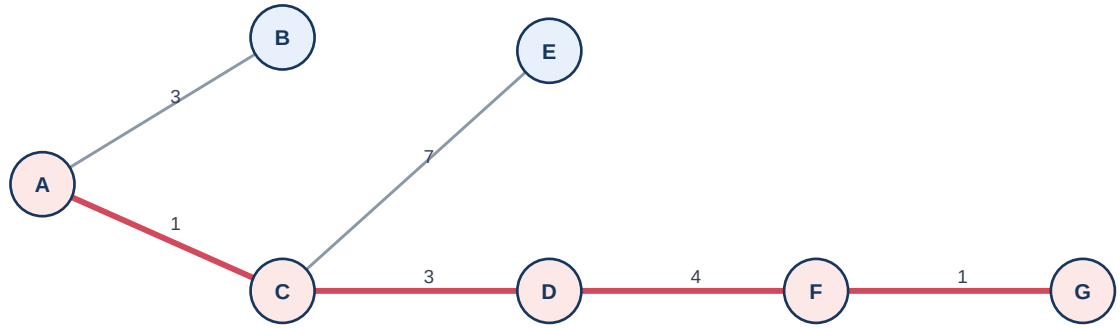
Aim: Implement A* search to find a least-cost path using heuristic information.

Source note: The supplied Part B uses Python, a graph with weighted edges, heuristic values and a start/goal pair. The cleaned version also initializes the start priority correctly and ignores outdated priority-queue entries.

Core concept

A* selects the node with the smallest evaluation value: **f(n) = g(n) + h(n)**. g(n) is the actual cost from the start; h(n) estimates the remaining cost to the goal. An admissible heuristic never overestimates the true remaining cost.

A* weighted graph (edge labels are costs)



Final path: A → C → D → F → G. **Total cost:** 1 + 3 + 4 + 1 = 9. A* chooses the next node using f(n) = g(n) + h(n), not simply the shortest-looking branch.

Practical code

```
import heapq

def a_star(graph, h, start, goal):
    # Priority queue stores (f-cost, node)
    open_list = [(h[start], start)]
    g = {start: 0}
    parent = {start: None}

    while open_list:
        f, current = heapq.heappop(open_list)

        # Ignore an outdated queue entry
        if f != g[current] + h[current]:
            continue

        if current == goal:
            path = []
            node = goal
            while node is not None:
                path.append(node)
                node = parent[node]
            path.reverse()
            return path, g[goal]

        for neighbour, cost in graph[current]:
            new_g = g[current] + cost

            if neighbour not in g or new_g < g[neighbour]:
                g[neighbour] = new_g
                parent[neighbour] = current
                new_f = new_g + h[neighbour]
                heapq.heappush(open_list, (new_f, neighbour))

    return None, float("inf")


graph = {
    "A": [("B", 3), ("C", 1)],
    "B": [],
    "C": [("D", 3), ("E", 7)],
    "D": [("F", 4)],
```

```
    "E": [],
    "F": [("G", 1)],
    "G": []
}

heuristic = {
    "A": 7, "B": 5, "C": 6, "D": 4,
    "E": 1, "F": 1, "G": 0
}

path, cost = a_star(graph, heuristic, "A", "G")
print("Shortest path:", " -> ".join(path) if path else "Not found")
print("Total cost:", cost)
```

Code explanation

• **heapq:** implements a min-priority queue, so the node with the smallest f-cost is removed first.

• **g dictionary:** stores the cheapest known cost from the start to each node.

• **parent dictionary:** remembers the previous node, allowing the final path to be reconstructed.

• **Expansion:** for every neighbour, calculate new_g. If it is a better route, update g and parent, then push new_g + h(neighbour) into the queue.

• **Goal:** when the goal is removed as the best current queue entry, follow parent links backwards and reverse the list.

Easy memory line

A* = **Actual + Estimated**. f = g + h. g = travelled cost; h = guess to goal; f = total estimated cost.

Input and expected output

Input: Graph: A-B(3), A-C(1), C-D(3), C-E(7), D-F(4), F-G(1). Start A, goal G.

Expected output: Shortest path: A -> C -> D -> F -> G. Total cost: 9? (Check the edge costs in the supplied graph: 1 + 3 + 4 + 1 = **9**.)

How to run: *Save as exp3.py and run:* ***python exp3.py****. The code uses only Python's built-in heapq module.*

Cost correction: *The Part B graph's displayed edge costs produce a total of 9 for A-C-D-F-G. Do not use 7 for this Part B program.*

EXPERIMENT 04

SINGLE LAYER PERCEPTRON: AND AND OR GATES

Aim: Build a single-layer perceptron using the perceptron learning rule.

Source note: The supplied Part B implements a step activation, trains weights and bias, and demonstrates AND and OR truth tables.

Core concept

A perceptron calculates net = x1*w1 + x2*w2 + bias. The step function returns 1 when net is at least zero; otherwise it returns 0. Training adjusts weights and bias using error = target - output.

Practical code

```
def step(x):
    return 1 if x >= 0 else 0

def train_perceptron(data, target):
    w1 = w2 = bias = 0.0
    rate = 0.1

    for epoch in range(20):
        errors = 0
        for i in range(len(data)):
            net = data[i][0] * w1 + data[i][1] * w2 + bias
            output = step(net)
            error = target[i] - output

            w1 += rate * error * data[i][0]
            w2 += rate * error * data[i][1]
            bias += rate * error

            if error != 0:
                errors += 1

        if errors == 0:
            break

    return w1, w2, bias

def test_perceptron(data, w1, w2, bias):
    return [
        step(x1 * w1 + x2 * w2 + bias)
        for x1, x2 in data
    ]

# Truth table for two inputs
X = [[0, 0], [0, 1], [1, 0], [1, 1]]

for gate, target in [
    ("AND", [0, 0, 0, 1]),
    ("OR",  [0, 1, 1, 1])
]:
    w1, w2, bias = train_perceptron(X, target)
    output = test_perceptron(X, w1, w2, bias)

    print("\\n", gate, "GATE")
    print("X1  X2  Output")
    for i in range(len(X)):
        print(X[i][0], "  ", X[i][1], "  ", output[i])
```

Code explanation

• **step(x):** the activation function converts the weighted sum into binary output 0 or 1.

• **train_perceptron:** starts weights and bias at zero, checks each training row, calculates the error and updates parameters using the learning rate.

• **Stopping:** if an epoch has no errors, training stops early. Otherwise it runs up to 20 epochs.

• **test_perceptron:** applies the learned weights to each input combination and returns predictions.

• **AND / OR:** the same network is trained twice with different target lists, so it learns each truth table separately.

Easy memory line

Perceptron loop: **Weighted sum -> Step activation -> Error -> Update weights/bias**.

Input and expected output

Input: Four input pairs: (0,0), (0,1), (1,0), (1,1). Targets: AND = 0,0,0,1; OR = 0,1,1,1.

Expected output: AND: 0, 0, 0, 1. OR: 0, 1, 1, 1.

How to run: *Save as exp4.py and run:* ***python exp4.py****. No external library is required.*

EXPERIMENT 05

MULTI-LAYER PERCEPTRON: SUPERVISED LEARNING

Aim: Implement a supervised learning algorithm using a multilayer perceptron (MLP).

Source note: The supplied Part B uses study hours and attendance to classify a student as PASS or FAIL. It has two input neurons, two hidden neurons and one output neuron.

Core concept

An MLP is a feedforward neural network with input, hidden and output layers. It learns from labelled examples. Forward propagation produces a prediction; backpropagation calculates gradients and updates weights to reduce prediction error.

Practical code

```
import math

def sigmoid(x):
    # Avoid overflow for unusually large values
    if x < -60:
        return 0.0
    if x > 60:
        return 1.0
    return 1 / (1 + math.exp(-x))

# Training data: [study hours, attendance]
X = [
    [1, 40], [2, 45], [2, 50], [3, 55],
    [3, 60], [4, 65], [4, 70], [5, 75],
    [5, 80], [6, 85], [6, 90], [7, 95]
]
Y = [0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1]

# Normalize both input features to a small range
X = [[hours / 10, attendance / 100] for hours, attendance in X]

# Two input neurons, two hidden neurons, one output neuron
w1 = w2 = w3 = w4 = w5 = w6 = 0.5
b1 = b2 = b3 = 0.1
rate = 0.5

for epoch in range(5000):
    total_error = 0

    for i in range(len(X)):
        x1, x2 = X[i]
        target = Y[i]

        # Forward propagation
        h1 = sigmoid(x1*w1 + x2*w2 + b1)
        h2 = sigmoid(x1*w3 + x2*w4 + b2)
        output = sigmoid(h1*w5 + h2*w6 + b3)

        error = target - output
        total_error += error ** 2

        # Backpropagation
        out_grad = error * output * (1 - output)
        h1_grad = h1 * (1 - h1) * out_grad * w5
        h2_grad = h2 * (1 - h2) * out_grad * w6

        w5 += rate * out_grad * h1
        w6 += rate * out_grad * h2
        b3 += rate * out_grad

        w1 += rate * h1_grad * x1
        w2 += rate * h1_grad * x2
        b1 += rate * h1_grad

        w3 += rate * h2_grad * x1
        w4 += rate * h2_grad * x2
        b2 += rate * h2_grad

    if total_error < 0.01:
        break

print("MLP SUPERVISED LEARNING")
print("Training completed in", epoch + 1, "epochs")

unknown_data = [[2, 48], [5, 78], [7, 92]]
print("\\nPredictions for unknown data:")

for hours, attendance in unknown_data:
```

```
x1, x2 = hours / 10, attendance / 100
h1 = sigmoid(x1*w1 + x2*w2 + b1)
h2 = sigmoid(x1*w3 + x2*w4 + b2)
output = sigmoid(h1*w5 + h2*w6 + b3)
result = "PASS" if output >= 0.5 else "FAIL"

print("Study:", hours, "Attendance:", attendance,
      "Output:", round(output, 3), "Prediction:", result)
```

Code explanation

• **Training data:** X contains study hours and attendance. Y contains the correct label: 0 = FAIL and 1 = PASS.

• **Normalization:** study hours are divided by 10 and attendance by 100 so the inputs are in a smaller numerical range.

• **Sigmoid:** converts a neuron's weighted sum into a value between 0 and 1.

• **Forward propagation:** two hidden neurons (h1, h2) calculate values; the output neuron combines them to produce a PASS probability.

• **Error and gradients:** error = target - output. The output gradient and hidden gradients indicate how each weight contributed to the error.

• **Backpropagation:** weights and biases are adjusted using the learning rate. Training repeats for up to 5000 epochs or until total error is small.

• **Prediction:** unseen examples are passed through the trained network. Output >= 0.5 means PASS; otherwise FAIL.

Easy memory line

MLP = **Input -> Hidden -> Output**. Training = **Forward pass -> Error -> Backpropagation -> Weight update**.

Input and expected output

Input: The training examples are built into the code. Three unseen examples are tested: [2,48], [5,78], [7,92].

Expected output: The program prints the training completion epoch and PASS/FAIL predictions with output scores.

How to run: *Save as exp5.py and run:* ***python exp5.py****. Only the built-in math module is used.*

EXPERIMENT 06

KOHONEN SELF-ORGANIZING MAP (KSOM)

Aim: Implement an unsupervised Kohonen Self-Organizing Map for clustering.

Source note: The supplied Part B accepts vectors, number of clusters, learning rate and epochs. It chooses the closest weight vector and updates the winning cluster.

Core concept

KSOM is unsupervised: no target labels are supplied. For each input, calculate Euclidean distance to each cluster weight. The closest cluster is the winner (best matching unit). Update its weight using Wnew = Wold + alpha(X - Wold).

Practical code

```
import math

def distance(a, b):
    return math.sqrt(sum((a[i] - b[i]) ** 2 for i in range(len(a))))

# Enter vectors as space-separated numbers, for example: 1 2 3
n = int(input("Number of input vectors: "))
data = []

for i in range(n):
    vector = list(map(float, input("Vector " + str(i + 1) + ": ").split()))
    if data and len(vector) != len(data[0]):
        raise ValueError("All vectors must have the same number of values.")
    data.append(vector)

clusters = int(input("Number of clusters: "))
rate = float(input("Learning rate (0 to 1): "))
epochs = int(input("Number of epochs: "))

if clusters < 1 or clusters > n:
    raise ValueError("Clusters must be between 1 and number of vectors.")
if not 0 < rate <= 1:
    raise ValueError("Learning rate must be greater than 0 and at most 1.")

# First 'clusters' vectors become initial weight vectors
weights = [data[i][:] for i in range(clusters)]

for epoch in range(epochs):
    for x in data:
        distances = [distance(x, w) for w in weights]
        winner = distances.index(min(distances))

        # Update the winning neuron's weights
        for j in range(len(x)):
            weights[winner][j] += rate * (x[j] - weights[winner][j])

    rate *= 0.9

print("\\nFinal weights:")
for i, w in enumerate(weights):
    print("Cluster", i + 1, ":", [round(v, 3) for v in w])

print("\\nFinal clusters:")
for x in data:
    distances = [distance(x, w) for w in weights]
    winner = distances.index(min(distances))
    print(x, "-> Cluster", winner + 1)
```

Code explanation

• **distance(a,b):** calculates Euclidean distance, which measures how close two vectors are.

• **Initial weights:** the first 'clusters' input vectors are copied as starting cluster weights.

• **Winner:** calculate each distance and choose the index of the smallest one.

• **Weight update:** move the winning weight vector towards the current input by the learning rate.

• **Learning rate decay:** rate is multiplied by 0.9 after each epoch, making later updates smaller.

• **Final assignment:** each vector is assigned to whichever final weight vector is closest.

Easy memory line

KSOM = **Distance -> Winner -> Update**. No labels. Closest weight wins.

Input and expected output

Input: Example: 4 vectors [0 0 1 1], [1 0 0 0], [0 1 1 0], [0 0 0 1]; clusters = 2; rate = 0.5; epochs = 5.

Expected output: The program displays final cluster weights and the cluster assigned to each input vector.

How to run: *Save as exp6.py and run:* ***python exp6.py****. Enter each vector as space-separated numbers. All vectors must have the same length.*

EXPERIMENT 07

ASSOCIATIVE MEMORY: PART B CODE IMPLEMENTS BAM

Aim: Implement an associative memory network. Important: the Part B code in the supplied PDF is specifically a Bidirectional Associative Memory (BAM) implementation.

Source note: The experiment's theory discusses auto-associative memory, but the Part B program is titled 'BIDIRECTIONAL ASSOCIATIVE MEMORY (BAM)' and uses 4-bit input patterns paired with 3-bit output patterns. This handbook follows the actual Part B code.

Core concept

BAM associates one pattern X with a different pattern Y and supports recall in both directions. Hebbian learning builds the weight matrix using W = X-transpose multiplied by Y. For recall, multiply an input pattern by W and apply a bipolar activation function that returns +1 or -1.

Practical code

```
# Bidirectional Associative Memory (BAM)
# No external library is required.

X = [
    [ 1,  1,  1,  1], [ 1,  1,  1, -1],
    [ 1,  1, -1,  1], [ 1,  1, -1, -1],
    [ 1, -1,  1,  1], [ 1, -1,  1, -1],
    [ 1, -1, -1,  1], [ 1, -1, -1, -1],
    [-1,  1,  1,  1], [-1,  1,  1, -1]
]

Y = [
    [ 1,  1,  1], [ 1,  1, -1],
    [ 1, -1,  1], [ 1, -1, -1],
    [-1,  1,  1], [-1,  1, -1],
    [-1, -1,  1], [-1, -1, -1],
    [ 1,  1,  1], [ 1,  1, -1]
]

# Hebbian learning: W = transpose(X) multiplied by Y
rows, cols = len(X[0]), len(Y[0])
W = [[0 for _ in range(cols)] for _ in range(rows)]

for x, y in zip(X, Y):
    for i in range(rows):
        for j in range(cols):
            W[i][j] += x[i] * y[j]

def activate(values):
    return [1 if value >= 0 else -1 for value in values]

def multiply(vector, matrix):
    return [
        sum(vector[i] * matrix[i][j] for i in range(len(vector)))
        for j in range(len(matrix[0]))
    ]

test_inputs = [
    [ 1,  1,  1,  1],
    [ 1,  1,  1, -1],
    [ 1,  1, -1,  1],
    [ 1, -1,  1,  1],
    [-1,  1,  1,  1]
]

expected = [
    [ 1,  1,  1],
    [ 1,  1, -1],
    [ 1, -1,  1],
    [-1,  1,  1],
    [ 1,  1,  1]
]

print("BIDIRECTIONAL ASSOCIATIVE MEMORY (BAM)")
print("\\nWeight matrix:")
for row in W:
    print(row)

errors = 0
print("\\nTesting (X -> Y):")

for i, pattern in enumerate(test_inputs):
    net = multiply(pattern, W)
```

```
    recalled = activate(net)
    correct = recalled == expected[i]

    print("\\nInput:", pattern)
    print("Recalled output:", recalled)
    print("Expected output:", expected[i])
    print("Result:", "Correct" if correct else "Incorrect")

    if not correct:
        errors += 1

print("\\nTotal tests:", len(test_inputs))
print("Errors:", errors)
print("Error rate:", round(errors / len(test_inputs) * 100, 2), "%")

# Reverse recall demonstrates the bidirectional association Y -> X
print("\\nReverse recall for first stored pair (Y -> X):")
reverse_net = multiply(Y[0], [list(row) for row in zip(*W)])
print("Input Y:", Y[0])
print("Recalled X:", activate(reverse_net))
```

Code explanation

• **X and Y:** X stores ten 4-element input patterns; Y stores their corresponding 3-element output patterns.

• **Weight matrix:** start with zeros, then add x[i] * y[j] for every paired training row. This is Hebbian correlation learning.

• **Activation:** positive or zero net input becomes +1; negative net input becomes -1.

• **multiply:** performs vector-matrix multiplication without NumPy, so the practical code has no external dependency.

• **Testing:** each test input is multiplied by W, converted to bipolar output and compared with its expected output. The error percentage is displayed.

• **Reverse recall:** the final section transposes W and demonstrates the Y-to-X direction, which is why the network is bidirectional.

Easy memory line

BAM = **Two different patterns**. Learn with W = X^T Y. Recall forward with XW; reverse with YW^T.

Input and expected output

Input: Training patterns are built into the program. Five test inputs are evaluated automatically.

Expected output: The program prints the weight matrix, recalled output, expected output, correctness and error rate. Some test pairs may be recalled incorrectly because associative memories have limited capacity and interference.

How to run: *Save as exp7.py and run:* ***python exp7.py****. The cleaned version uses pure Python; NumPy is not needed.*

Exam clarification: *The Part B code is BAM, which associates different input and output patterns. The theory also discusses auto-associative memory, but this code is not the auto-associative example.*

EXPERIMENT 08

FUZZY MEMBERSHIP FUNCTIONS

Aim: Calculate membership values using triangular, trapezoidal and Gaussian membership functions.

Source note: The supplied Part B uses ten sample values from 0 to 90 and calculates three membership values for each.

Core concept

A membership function maps an input to a degree between 0 and 1. 0 means no membership; 1 means full membership; values between them mean partial membership.

Practical code

```
import math

def triangular(x, a, b, c):
    if x <= a or x >= c:
        return 0.0
    if x == b:
        return 1.0
    if x < b:
        return (x - a) / (b - a)
    return (c - x) / (c - b)

def trapezoidal(x, a, b, c, d):
    if x <= a or x >= d:
        return 0.0
    if b <= x <= c:
        return 1.0
    if x < b:
        return (x - a) / (b - a)
    return (d - x) / (d - c)

def gaussian(x, center, sigma):
    return math.exp(-0.5 * ((x - center) / sigma) ** 2)

data = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]

print("FUZZY MEMBERSHIP FUNCTIONS")
print("{:<8} {:<12} {:<12} {:<12}".format(
    "Input", "Triangular", "Trapezoidal", "Gaussian"))

for x in data:
    tri = triangular(x, 20, 50, 80)
    trap = trapezoidal(x, 10, 30, 60, 80)
    gau = gaussian(x, 50, 20)
    print("{:<8} {:<12.3f} {:<12.3f} {:<12.3f}".format(
        x, tri, trap, gau))

print("All membership values calculated successfully.")
```

Code explanation

• **triangular(x,a,b,c):** membership rises linearly from a to peak b, then falls linearly to c.

• **trapezoidal(x,a,b,c,d):** membership rises from a to b, stays at 1 from b to c, then falls to zero at d.

• **gaussian(x,center,sigma):** produces a smooth bell-shaped curve. center is the peak; sigma controls its width.

• **Loop:** each sample value is passed to all three functions and the results are printed in columns.

Easy memory line

Triangle = one peak. Trapezoid = flat top. Gaussian = smooth bell. Every membership value is between **0 and 1**.

Input and expected output

Input: Sample data: 0, 10, 20, 30, 40, 50, 60, 70, 80, 90.

Expected output: A table containing Input, Triangular, Trapezoidal and Gaussian membership values.

How to run: *Save as exp8.py and run:* ***python exp8.py****. No external library is required.*

EXPERIMENT 09

FUZZY CONTROL SYSTEM SIMULATION

Aim: Simulate a fuzzy controller that recommends fan speed using room temperature and humidity.

Source note: The supplied Part B fuzzifies temperature and humidity, applies nine rules and calculates fan speed using weighted-average defuzzification.

Core concept

A fuzzy controller converts crisp inputs into membership values (fuzzification), evaluates IF-THEN rules (inference), and converts the fuzzy result into one crisp output (defuzzification).

Practical code

```
def temperature_membership(temp):
    if temp <= 20:
        return 1.0, 0.0, 0.0
    elif temp < 30:
        return (30-temp)/10, (temp-20)/10, 0.0
    elif temp == 30:
        return 0.0, 1.0, 0.0
    elif temp < 40:
        return 0.0, (40-temp)/10, (temp-30)/10
    return 0.0, 0.0, 1.0

def humidity_membership(humidity):
    if humidity <= 30:
        return 1.0, 0.0, 0.0
    elif humidity < 50:
        return (50-humidity)/20, (humidity-30)/20, 0.0
    elif humidity == 50:
        return 0.0, 1.0, 0.0
    elif humidity < 70:
        return 0.0, (70-humidity)/20, (humidity-50)/20
    return 0.0, 0.0, 1.0

temp = float(input("Enter room temperature (10-40 C): "))
humidity = float(input("Enter humidity (0-100%): "))

if not 0 <= humidity <= 100:
    raise ValueError("Humidity must be between 0 and 100.")

cold, comfortable, hot = temperature_membership(temp)
dry, normal, humid = humidity_membership(humidity)

print("\\nFuzzification results:")
print("Cold:", round(cold, 2), "Comfortable:", round(comfortable, 2),
      "Hot:", round(hot, 2))
print("Dry:", round(dry, 2), "Normal:", round(normal, 2),
      "Humid:", round(humid, 2))

# Each rule is (rule strength, fan speed)
rules = [
    (min(cold, dry), 20), (min(cold, normal), 25),
    (min(cold, humid), 30), (min(comfortable, dry), 40),
    (min(comfortable, normal), 50), (min(comfortable, humid), 60),
    (min(hot, dry), 70), (min(hot, normal), 85),
    (min(hot, humid), 100)
]

# Weighted-average defuzzification
numerator = sum(strength * speed for strength, speed in rules)
denominator = sum(strength for strength, speed in rules)
fan_speed = numerator / denominator if denominator else 0

print("\\nFuzzy control output:")
print("Recommended fan speed:", round(fan_speed, 2), "%")

if fan_speed < 35:
    print("Fan status: LOW")
elif fan_speed < 65:
    print("Fan status: MEDIUM")
else:
    print("Fan status: HIGH")

print("Fuzzy control simulation completed successfully.")
```

Code explanation

• **temperature_membership:** returns membership in Cold, Comfortable and Hot sets.

• **humidity_membership:** returns membership in Dry, Normal and Humid sets.

• **Fuzzification:** the entered temperature and humidity become degrees of membership in their linguistic categories.

• **Rule base:** nine combinations map temperature/humidity categories to fan speeds from 20% to 100%. min() gives each rule's strength.

• **Weighted average:** multiply each rule strength by its fan speed, add the products, then divide by the sum of strengths.

• **Fan status:** the final numeric speed is labelled LOW, MEDIUM or HIGH.

Easy memory line

Fuzzy controller flow: **Input -> Fuzzification -> Rules/Inference -> Defuzzification -> Output**.

Input and expected output

Input: Enter room temperature in Celsius and humidity percentage. Example: temperature = 32, humidity = 65.

Expected output: Membership values, recommended fan speed percentage and LOW/MEDIUM/HIGH status.

How to run: *Save as exp9.py and run:* ***python exp9.py****. Humidity must be between 0 and 100.*

EXPERIMENT 10

GENETIC ALGORITHM OPTIMIZATION

Aim: Use a genetic algorithm to maximize f(x) = x squared for integer x from 0 to 31.

Source note: The supplied Part B uses a population of 10, 20 generations, 5-bit chromosomes, fitness evaluation, selection, one-point crossover and mutation.

Core concept

A genetic algorithm is an optimization method inspired by natural selection. It maintains a population of candidate solutions and improves them through selection, crossover and mutation.

Practical code

```
import random

def fitness(x):
    return x * x

POPULATION_SIZE = 10
GENERATIONS = 20
MUTATION_RATE = 0.1

# Each chromosome is a 5-bit integer from 0 to 31
population = [random.randint(0, 31) for _ in range(POPULATION_SIZE)]
best = max(population, key=fitness)

print("GENETIC ALGORITHM: MAXIMIZE f(x) = x^2")
print("Initial population:", population)

for generation in range(GENERATIONS):
    population.sort(key=fitness, reverse=True)

    if fitness(population[0]) > fitness(best):
        best = population[0]

    print("Generation", generation + 1,
          "| Best x =", population[0],
          "| Fitness =", fitness(population[0]))

    # Selection: two fittest chromosomes
    parent1, parent2 = population[0], population[1]
    p1, p2 = format(parent1, "05b"), format(parent2, "05b")

    # One-point crossover
    point = random.randint(1, 4)
    child1 = p1[:point] + p2[point:]
    child2 = p2[:point] + p1[point:]

    children = []
    for child in [child1, child2]:
        bits = list(child)

        # Mutation: flip one randomly selected bit
        if random.random() < MUTATION_RATE:
            pos = random.randint(0, 4)
            bits[pos] = "1" if bits[pos] == "0" else "0"

        children.append(int("".join(bits), 2))

    # Elitism: preserve the two best; add offspring and fresh individuals
    population = population[:2] + children
    while len(population) < POPULATION_SIZE:
        population.append(random.randint(0, 31))

    best = max(population + [best], key=fitness)

print("\\nFinal result")
print("Best solution x =", best)
print("Maximum fitness =", fitness(best))
print("Target: maximize x^2, where 0 <= x <= 31")
print("Genetic algorithm completed successfully.")
```

Code explanation

- **Chromosome:** each integer from 0 to 31 is represented by a 5-bit binary string.
- **Fitness:** fitness(x) = x*x. A larger value is better because this is a maximization problem.
- **Selection:** the two candidates with the highest fitness are selected as parents.

• **Crossover:** choose a random split point and exchange the right-hand parts of the two parent bit strings to create children.

• **Mutation:** with probability 0.1, flip one randomly selected bit in a child to introduce variation.

• **Elitism:** preserve the two best candidates when forming the next generation; fill remaining places with offspring and random candidates.

• **Termination:** repeat for 20 generations and print the best solution found.

Easy memory line

GA cycle = **Population -> Fitness -> Selection -> Crossover -> Mutation -> New generation**.

Input and expected output

Input: No manual input. Population and genetic parameters are defined at the top of the program.

Expected output: Best x and maximum fitness found. The theoretical optimum is x = 31 and fitness = 961; a random run may or may not reach it.

How to run: *Save as exp10.py and run:* ***python exp10.py****. Results can vary because the algorithm uses randomness.*

Important: *Genetic algorithms use randomness, so the best result can vary. The theoretical maximum is x = 31 and f(x) = 961.*